

如何分三步实施微前端

大纲

微前端（概念） 认知史

为什么需要微前端

调研到落地实践

总结和规划

主要内容

多应用集成 - Qiankun (乾坤)

单体拆分 - Federation 的探索

我的认知史



微前端不是银弹

它并没有多么高深莫测

为什么需要



业务价值

内部应用太多
UI 风格不一致
多应用操作断层



工程价值

统一管理
模块拆分 多人合作
发布提速

可能遇到的问题？

全局的样式冲突

Shadow DOM | Css Scope

JS 污染

tc39/proposal-realms

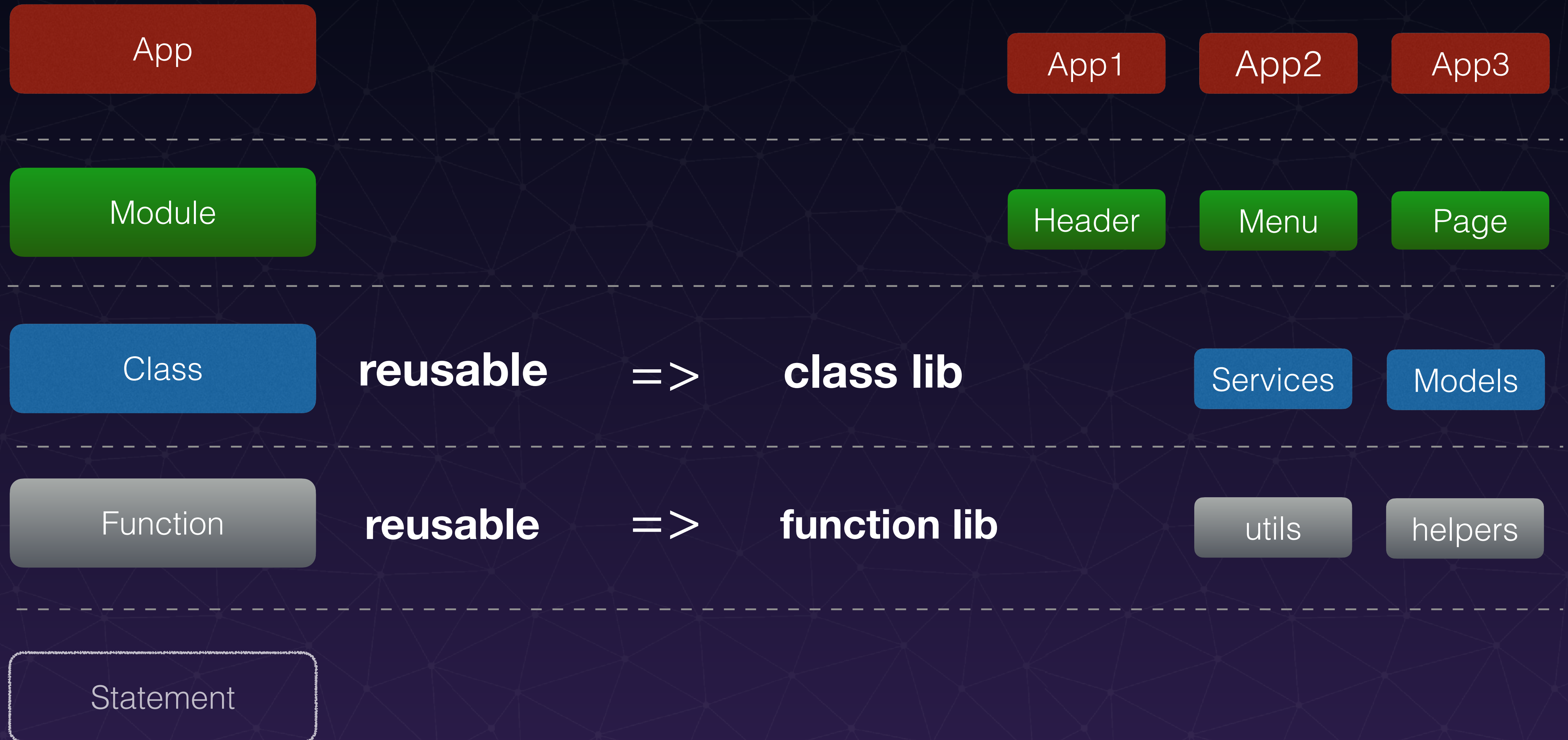
某些库多版本

Externals => DLL

iframe ?

想清楚粒度问题

程序的粗细粒度



应用集成

Single Spa

VS

Qiankun

SystemJs or other

single-spa + sandbox +
import-html-entry

应用集成

主应用

```
const apps = [  
  {  
    name: 'xyz',  
    entry: '服务地址',  
    container: '#subApp',  
    activeRule: '/xyz',  
    props: { menuService }  
  }  
]  
  
registerMicroApps(apps)  
  
setDefaultMountApp('xyz')  
  
start({  
  prefetch: 'all'  
})
```

子应用

```
export async function bootstrap() {}  
  
export async function mount(props) {  
  renderApp()  
}  
  
export async function unmount() {  
  ReactDOM.unmountComponentAtNode(document.getElementById('xyz-root'))  
}
```

+ webpack 的设置 (将子应用打成一个 lib)

遇到的实际问题

重复配置

将打包配置抽成解决方案 (highway 插件)

DLL

勿忘设置 DLL 的 libraryTarget (sandbox 源码)

AntD Modal

getContainer 指定局部渲染的节点

父子通讯

props 在子 mount 时传入

多Store 共存

利用 props 传入 context | storeKey

应用集成

简单模式

整页覆盖渲染 + 导航器浮层

精细模式

提供 Content 渲染区域给子应用

- 协定菜单数据结构，注给子应用 menuService
- 提供自定义 Menu、Header、Footer 的自定义接口
- 注入统一请求库，利用 LRU 策略缓存接口。

单体拆分

<https://zh-hans.single-spa.js.org/docs/separating-applications>

	Monorepo	NPM包	动态加载模块
搭建难度	简单	中等	困难
代码是否独立	No	No	✓
分开构建	No	✓	✓
分别部署	No	✓	✓

单体拆分 - 动态加载

qiankun@1.x ?

single-spa parcel + systemjs ?

qiankun@2.x ?

单体拆分



Tobias Koppers
@wSokra

I was asked multiple times about How to Micro-Frontend with webpack, but I had no good answer so far. Now the (hyped) Module Federation is part of webpack v5.0.0-beta.16 and I finally have a good answer to this problem.

[翻译推文](#)



webpack/webpack

A bundler for javascript and friends. Packs many modules into a few bundled assets. Code Splitting allows for loading ...

[github.com](#)

上午6:12 · 2020年5月6日 · [Twitter Web App](#)

Federation

Federation

Externals 简单粗暴，无法处理共存的版本

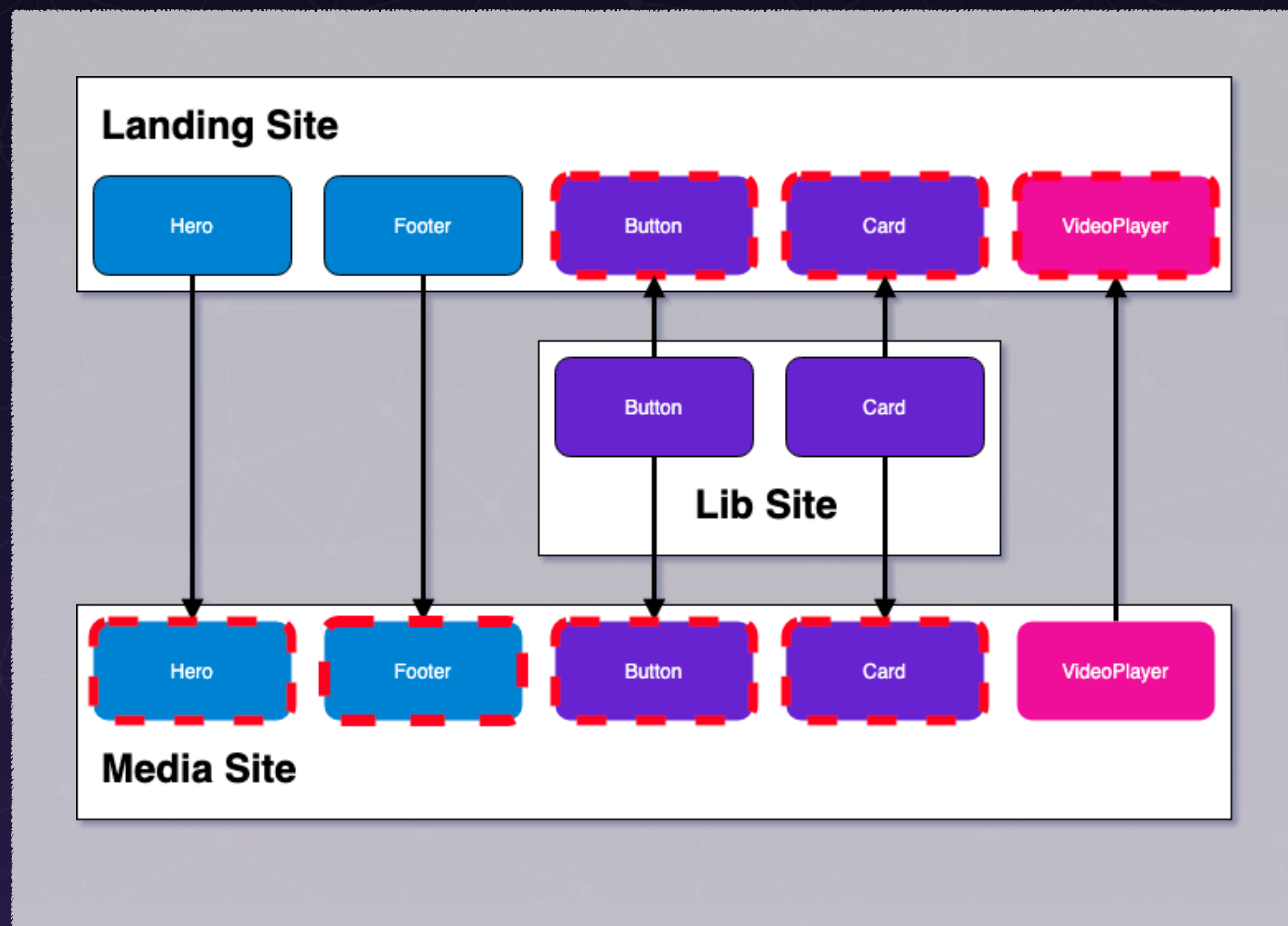
DLL 打包进模块，存在冗余的代码

Federation 可以对依赖库做到版本控制、自动加载缺失的 vendor

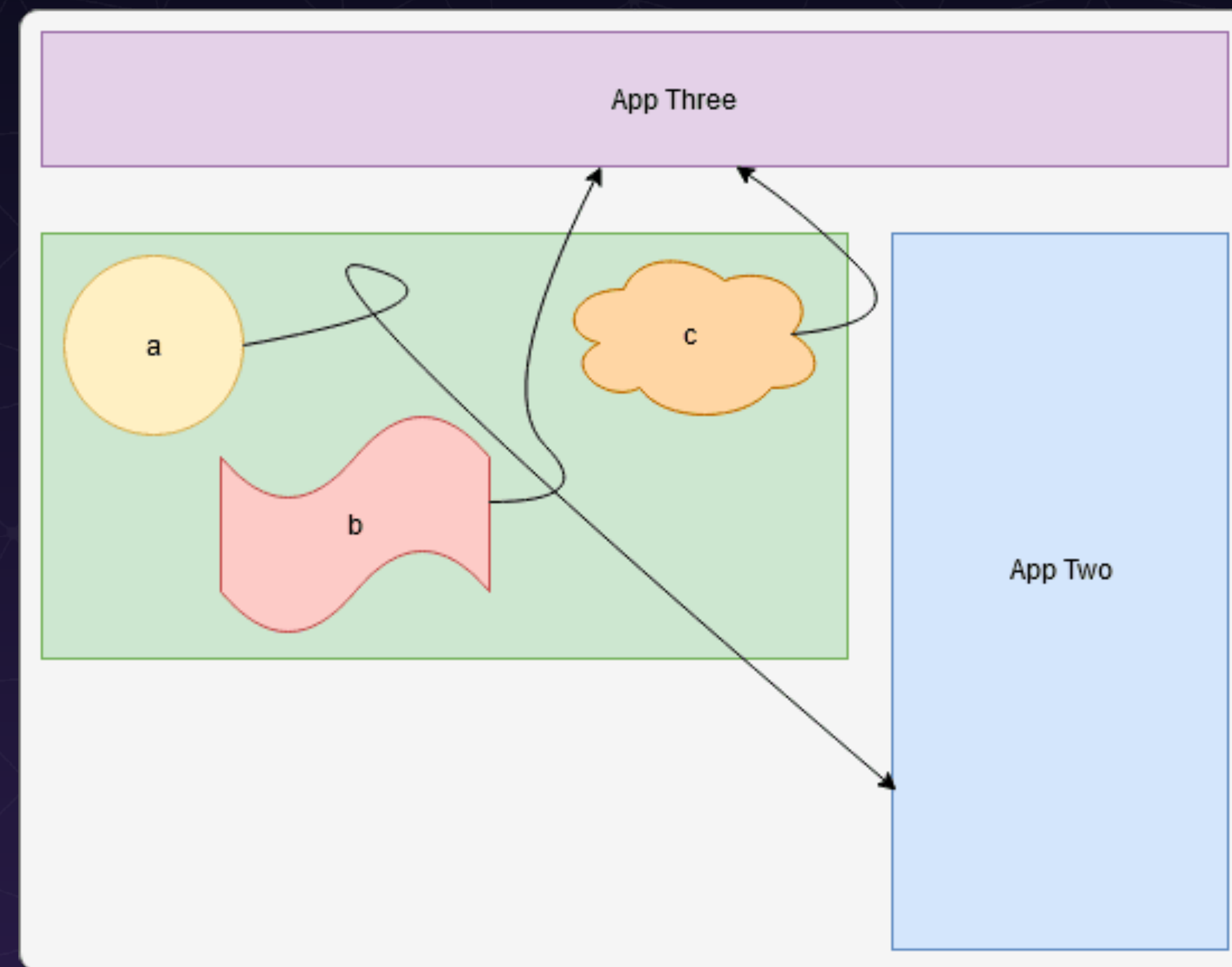
Federation 支持远程加载，解决了 npm link 在开发时不便利

Federation

页面间共享



单页模块间共享




```

new ModuleFederationPlugin({
  name: 'application_a',
  library: { type: 'var', name: 'application_a' },
  filename: 'remoteEntry.js',
  exposes: {
    'SayHelloFromA': './src/app',
  },
  remotes: {
    'application_b': 'application_b',
  },
  shared: ['react', 'react-dom']
})

```

```

<head>
  <script src="http://localhost:3002/remoteEntry.js"></script>
</head>

```

```
import SayHelloFromB from 'application_b/SayHelloFromB'
```

```

new ModuleFederationPlugin({
  name: 'application_b',
  library: { type: 'var', name: 'application_b' },
  filename: 'remoteEntry.js',
  exposes: {
    'SayHelloFromB': './src/app',
  },
  remotes: {
    'application_a': 'application_a',
  },
  shared: ['react', 'react-dom']
})

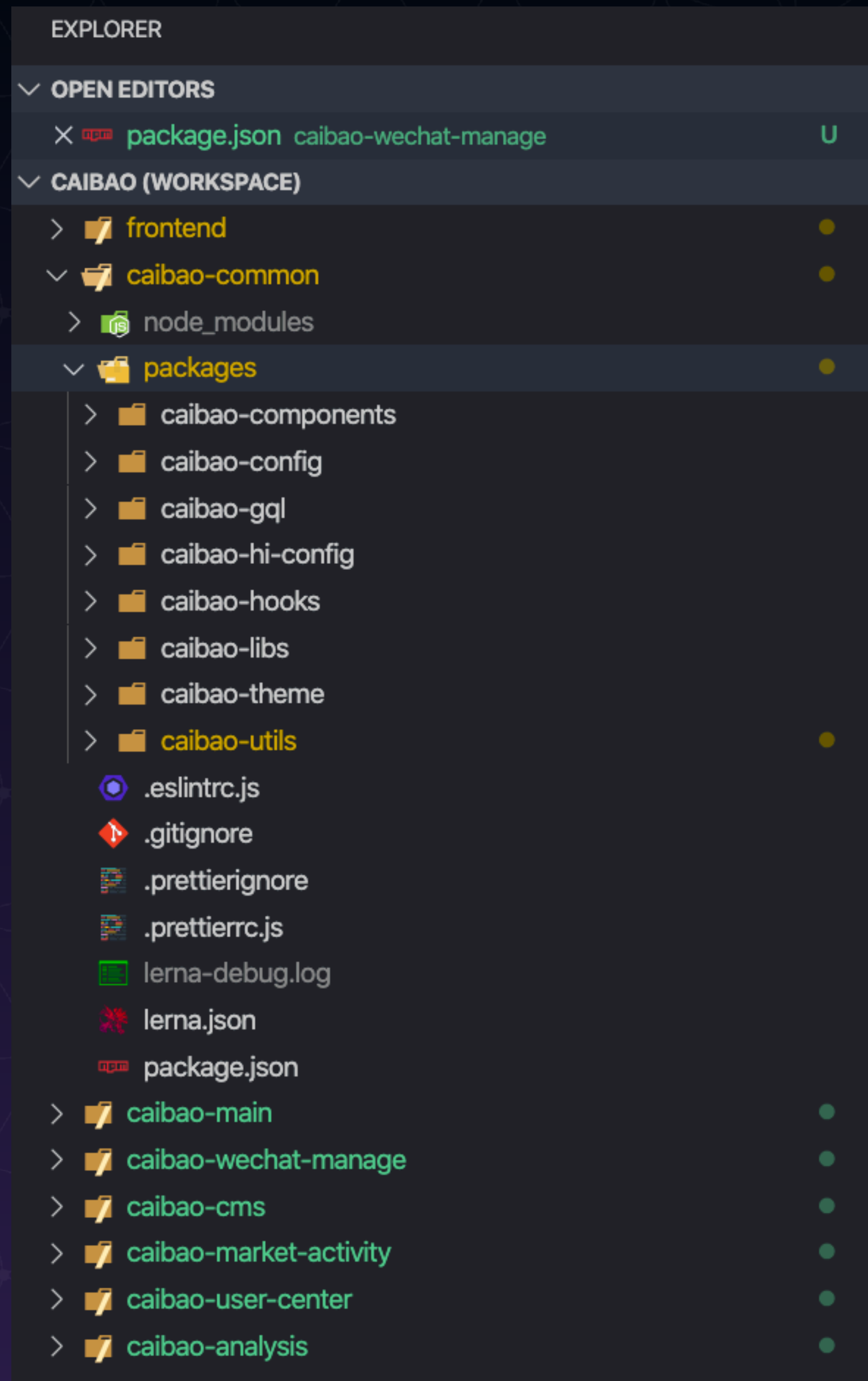
```

```

<head>
  <script src="http://localhost:3001/remoteEntry.js"></script>
</head>

```

```
import SayHelloFromA from 'application_b/SayHelloFromA'
```



公共包抽离

业务域划分模块



Federation
Shared

业务块划分也是一个问题

养成好习惯 - 按照业务领域划分页面

发布问题

简单模式

精细模式

精细模式

平滑上线

版本机制

index.[hash].html

配置中心

remoteEntry.[hash].js

发布次序

主应用最后发布

微前端不是银弹

适合的才是最好的

规划

开发入驻

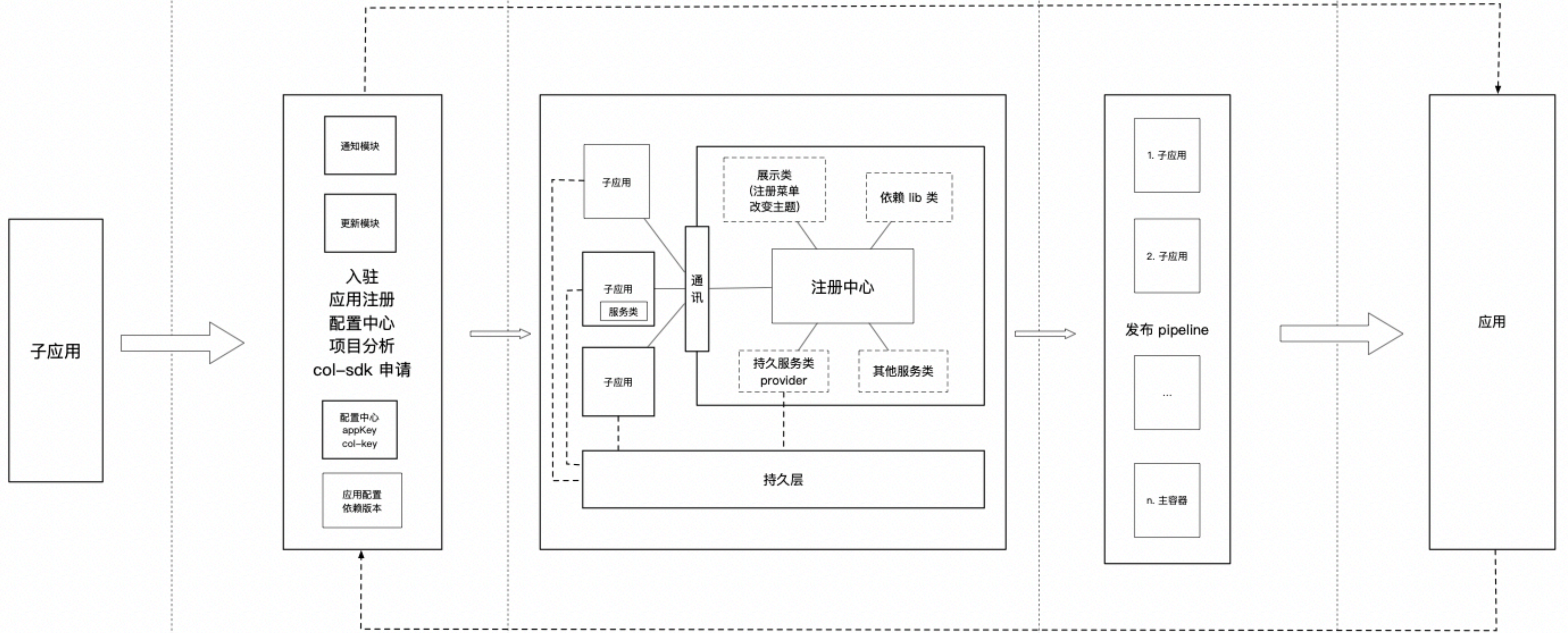
平台中心

运行时架构

发布平台

用户端

Push: 动态更新、增量发布



配置获取、依赖获取

发布方案还是简单模式

灰度方案也需要集成

通知机制开发

多应用继承，对监控平台也造成了开发量（域名变 path）

Thank you !